TP N°6: TDD - Test Driven Development



Hoja de Estilos - Estándares y convenciones a seguir para trabajar con las implementaciones a desarrollar

User Story implementada: Comprar entradas
EcoHarmony Park App

GRUPO 5
DOCENTES:

- Ing. Judith Meles
- Ing. Valeria Abdala
- Salvador Barbera

INTEGRANTES:

- Brocanelli, Gaspar 400989
- Comas, Lautaro 96586
- Fassi, Manuel 98518
- Gaitán Acevedo, Baltasar 401077
- Hermoza, Bianca Xiomara 95578
- Imoberdorf, Martín José 91570
- Koroch, Matías Adolfo 96369
- Morellato, Mayra 97325
- Romero Peñaloza, María Lucía 95694
- Salinas, Abril 94243
- Sanchez, Mateo 91785
- Tissera, Luciano 99236
- Vilchez, Felipe 95178

FRONT

1. Stack Tecnológico

El frontend es una aplicación web de una sola página (SPA) construida con **React** y **TypeScript**, usando **Ionic React** como biblioteca de componentes visuales y **Vite** como herramienta de desarrollo y compilación. Además está configurada como **PWA** (aplicación web instalable). La comunicación con el backend se realiza mediante la función nativa **fetch().**



2. Organización del código

```
frontend/
|-- index.html
|-- package.json
|-- vite.config.ts
|-- tsconfig.json  (y tsconfig.node.json)
|-- vercel.json
|-- ionic.config.json
|-- .prettierrc
|-- .env.example
|-- README.md
|-- public/
|   |-- icons/          (iconos de la PWA)
|   `-- images/         (fondo.jpg)
`-- src/
    |-- main.tsx
    |-- App.tsx
    |-- vite-env.d.ts
    |-- components/
    |   |-- Encabezado.tsx
    |   |-- EstadoConexion.tsx
    |   `-- Menu.tsx
    |-- hooks/
    |   |-- useInstalarApp.ts
    |   `-- useOnline.ts
    |-- pages/
    |   |-- Inicio.tsx
    |   |-- Actividades.tsx
    |   |-- ComprarEntradas.tsx
    |   |-- CompraFinalizada.tsx
    |   `-- Mapa.tsx
    |-- services/
    |   |-- api.ts
    |   |-- reglasCompra.ts
    |   |-- reglasCompra.test.ts
    |   |-- tipos.ts
    |   `-- ultimaCompra.ts
    |-- data/
    |   `-- actividades.ts
    `-- theme/
        |-- variables.css
        `-- global.css
```

El código se separa por responsabilidad.

- La lógica de la compra (reglas, precios y comunicación con la API) está en **services/**
- **main.tsx / App.tsx:** punto de entrada y definición de rutas
- **pages/:** una página por ruta. Las que implementan la user story son ComprarEntradas.tsx (formulario de compra) y CompraFinalizada.tsx (retorno desde Mercado Pago). Inicio, Actividades y Mapa complementan la aplicación.
- **components**/: menú lateral e indicador de conexión (En línea / Sin conexión).
- **hooks/:** useOnline informa el estado de la conexión y useInstalarApp gestiona la instalación de la PWA.
- **services/:** api.ts (llamadas HTTP al backend), reglasCompra.ts (validaciones, tarifas y calendario), tipos.ts (interfaces compartidas) y ultimaCompra.ts (guarda la última compra para recuperarla al volver de Mercado Pago).
- **theme/:** variables.css (colores de Ionic) y global.css (estilos propios de la aplicación)

3. Librerías usadas
   No se utiliza ninguna librería externa para las llamadas HTTP, para el manejo de estado ni para los estilos: se emplean las APIs nativas del navegador (fetch, sessionStorage, Intl) y los hooks de React.

4. Styling
   El estilo se resuelve con CSS puro en dos archivos globales (variables.css y global.css) sobre el sistema de componentes de Ionic, que se personaliza mediante variables CSS (--ion-color-*). No se utilizan preprocesadores ni frameworks de CSS.

Técnica	Ejemplos de uso	Propósito
Variables de tema	--ion-color-primary, --eh-forest,--eh-lime, --eh-radius	Centralizan la paleta del parque (#386641, #6a994e, #a7c957, #f2e8cf, #bc4749) y los valores comunes.
Layout moderno	display: grid, display: flex, gap	Grid para la estructura de la compra (.compra-layout, .pasos-compra) y Flexbox para alinear elementos internos (.contador, .resumen-lateral-linea).
Diseño responsive	@media (max-width: 850px), @media (max-width: 640px)	A 850 px la compra pasa a una sola columna y el resumen deja de ser fijo; a 640 px se ajustan márgenes y tamaños
Interacciones y transiciones	transition, ion-button:hover, .paso-compra	Pequeños desplazamientos y cambios de sombra al pasar el cursor sobre botones y tarjetas.
Feedback visual de estado	.mensaje-error, .paso-compra.activo, .paso-compra.completado, .badge-conexion	Muestran errores de validación, el paso actual de la compra y si hay conexión
Movimiento reducido	@media (prefers-reduced-motion: reduce)	Desactiva animaciones y transiciones para quienes lo piden en su sistema.

5. Reglas de nombrado

Elemento	Convención	Ejemplos en el código
Componentes y páginas	PascalCase; el archivo `.tsx` lleva el mismo nombre	`ComprarEntradas`, `CompraFinalizada`, `Encabezado`, `EstadoConexion`
Hooks	camelCase con prefijo `use`	`useOnline`, `useInstalarApp`
Funciones y variables	camelCase, verbo + objeto	`cambiarCantidad`, `validarSolicitud`, `guardarUltimaCompra`, `soloDigitosEdad`
Estado de React	par `[valor, setValor]`	`[edades, setEdades]`, `[pasoActivo, setPasoActivo]`
Constantes	SCREAMING_SNAKE_CASE	`MAXIMO_ENTRADAS_POR_COMPRA`, `PRECIOS_PASE`, `ENDPOINT_COMPRAS`, `EMAIL_REGEX`
Tipos e interfaces	PascalCase	`SolicitudCompra`, `ResultadoCompra`, `TipoPase`, `Fecha`
Valores de tipos unión	MAYÚSCULAS (iguales a los enums del backend)	`'REGULAR'`, `'VIP'`, `'EFECTIVO'`, `'TARJETA'`
Campos enviados a la API	snake_case (igual que el backend)	`usuario_email`, `fecha_visita`, `tipo_pase`, `forma_pago`, `payment_id`
Archivos de servicios y datos	camelCase	`reglasCompra.ts`, `ultimaCompra.ts`, `api.ts`, `actividades.ts`
Archivos de pruebas	`*.test.ts` junto al módulo probado	`reglasCompra.test.ts`
Rutas	kebab-case	`/compra-finalizada`
Clave de sessionStorage	prefijo del proyecto + kebab-case	`ecoharmony:ultima-compra`

Clases CSS (kebab-case)
Todas las clases, identificadores HTML y variables CSS propias usan kebab-case. No se aplica una metodología estricta (como BEM): las clases se organizan por prefijo de componente y por estado.

Tipo	Convención	Ejemplos en el código
Prefijo de componente	Las clases de un mismo componente o sección comparten el primer término del nombre.	.compra-intro, .compra-card, .compra-layout, .compra-formulario; .resumen-lateral-linea, .resumen-lateral-total; .hero-inicio, .hero-acciones
Modificador (estado)	Clase adicional aplicada sobre el mismo elemento, sin guiones dobles.	.paso-compra.activo, .paso-compra.completado, .opcion-seleccionada
Utilitarias de	Clases provistas por Ionic, usadas tal	ion-padding, ion-no-padding,

Ionic	cual junto a las propias.	ion-text-center
Identificadores HTML	Anclas de navegación dentro de la página.	#paso-fecha, #paso-entradas, #paso-pago, #resumen-compra

6. Misceláneas

   El código TypeScript se enfoca en la funcionalidad moderna del navegador y de React

   - **Comunicación**: usa la función nativa fetch() y async/await para todas las llamadas a la API
   - **Estado**: se gestiona con hooks de React (useState, useMemo, useEffect).
   - **Validación**: se realiza antes de enviar la solicitud con validarSolicitud, que encadena validaciones con el operador ?? y devuelve el primer mensaje de error o null.
   - **Experiencia de usuario:** el botón muestra un spinner mientras se envía la compra y se deshabilita sin conexión. Los errores se muestran en un mensaje con role="alert". Si el pago es en efectivo se muestra una alerta de confirmación (useIonAlert); si es con tarjeta se redirige a la URL de Mercado Pago con window.location.assign
   - **Retorno de Mercado Pago:** CompraFinalizada lee `status` y `payment_id` de la URL y consulta al backend (confirmarPago) antes de mostrar el resultado. Contempla los estados: verificando, pago aprobado, pago pendiente y pago no completado.

BACK

1. Nomenclatura
   - **PascalCase (o UpperCamelCase):** se usa para nombrar todas las clases. Por ejemplo: ServicioCompra, ValidadorCompra, Tarifario, CalendarioParque, PasarelaMercadoPago.
   - **snake_case:** se utiliza para nombrar funciones, métodos, variables, parámetros y atributos. Por ejemplo: sumar_un_mes, confirmar_pago, usuario_email, monto_total. Los métodos internos de una clase llevan un guión bajo inicial: _validar_fecha, _notificar, _calcular_monto_total.
   - **SCREAMING_SNAKE_CASE:** se utiliza para las constantes (valores que no cambian) y los miembros de las enumeraciones. Por ejemplo: MAXIMO_ENTRADAS_POR_COMPRA, PRECIOS_BASE, FERIADOS_CERRADO, HORA_LIMITE_COMPRA_MISMO_DIA, URL_PREFERENCIAS, TipoPase.REGULAR.

2. Organización del código

```
backend/
|-- requirements.txt
|-- notas.txt
|-- app/
|   |-- main.py
|   |-- api/
|   |   |-- dependencias.py
|   |   |-- esquemas.py
|   |   `-- rutas_compras.py
|   |-- servicios/
|   |   |-- servicio_compra.py
|   |   |-- validador_compra.py
|   |   |-- tarifario.py
|   |   |-- calendario_parque.py
|   |   |-- mail_confirmacion.py
|   |   `-- plantillas/
|   |       |-- mail_confirmacion.html
|   |       `-- fila_visitante.html
|   |-- integraciones/
|   |   |-- mercado_pago.py
|   |   |-- email_smtp.py
|   |   `-- errores.py
|   |-- repositorios/
|   |   |-- repositorio_compras.py
|   |   `-- repositorio_usuarios.py
|   `-- modelos/
|       `-- modelos.py
`-- tests/
    |-- unitarias/
    |   |-- test_tarifario.py
    |   |-- test_calendario_parque.py
    |   |-- test_mercado_pago.py
    |   |-- test_email_smtp.py
    |   `-- test_mail_confirmacion.py
    |-- integracion/
    |   `-- test_comprar_entrada.py
    |-- sistema/
    |   `-- test_api_compras.py
    `-- aceptacion/
        `-- test_pruebas_de_usuario.py
```

El backend se organiza en capas:

- **main.py:** punto de entrada de la API. Crea la aplicación FastAPI («EcoHarmony Park API»), configura CORS, registra el router de compras y expone GET /salud.
- **api/:** capa HTTP. rutas_compras.py define los endpoints; esquemas.py los modelos Pydantic de entrada y salida; dependencias.py arma el servicio con la configuración del entorno, identifica al usuario y define la zona horaria del parque (America/Argentina/Cordoba)
- **servicios/:** reglas de negocio. ServicioCompra orquesta la compra; ValidadorCompra valida la solicitud; Tarifario calcula precios; CalendarioParque indica si el parque está abierto; mail_confirmacion.py arma el mail con su código QR a partir de las plantillas HTML de plantillas/.
- **integraciones/:** conexión con sistemas externos. PasarelaMercadoPago crea el link de pago y consulta pagos (API de Mercado Pago); ServicioEmailSMTP envía mails por SMTP; PasarelaPagoError representa los fallos de la pasarela.

Endpoints

Método y ruta	Función	Descripción	Códigos de respuesta
`POST /api/compras`	`comprar_entradas`	Registra la compra. Con tarjeta devuelve `url_pago` de Mercado Pago; en efectivo envía el mail de confirmación.	201, 401, 403, 422
`POST /api/compras/confirmar-pago`	`confirmar_pago`	Verifica con Mercado Pago el pago (`payment_id`) y, si fue aprobado, envía el mail una sola vez.	200, 401, 403, 422
`GET /salud`	`salud`	Comprobación de estado del servicio.	200

Estrategia de pruebas

Las pruebas están organizadas en cuatro niveles, de lo más específico a lo más general.

El total es de 113 casos de prueba.

Nivel	Archivo	Qué verifica	Casos
Unitarias	`test_tarifario.py`	Precio por tipo de pase y tramo de edad, incluidos los límites (0, 3, 4, 15, 16, 59, 60 y 120 años).	22
	`test_calendario_parque.py`	Cierre los lunes, el 25/12 y el 1/1.	5
	`test_mercado_pago.py`	Pasarela con HTTP simulado: creación de preferencia, referencia de compra, consulta de pago, errores y retorno automático.	7
	`test_email_smtp.py`	Envío por SMTP simulado: texto, HTML e imagen incrustada.	2
	`test_mail_confirmacion.py`	Código de entrada, QR, contenido del mail y subtotales por visitante.	5
Integración	`test_comprar_entrada.py`	`ServicioCompra` completo con repositorios, Mercado Pago y SMTP en memoria: validaciones, límites, montos, pago, mail y confirmación.	55
Sistema	`test_api_compras.py`	API HTTP con `TestClient`: códigos 401/403/422, CORS, `/salud` y configuración obligatoria.	13
Aceptación	`test_pruebas_de_usuario.py`	Las cuatro pruebas de usuario (PU1 a PU4) definidas en la user story.	4

3. Estructura de funciones

   Decoradores: se utilizan para registrar funciones como manejadores de solicitudes, asociándolas a una URL y a un método HTTP, de modo que la ruta queda declarada justo encima de la función que implementa la lógica

4. Control de errores

   Devolución de errores en formato JSON: el backend responde los fallos con un objeto JSON con el mensaje en el campo detail y un código HTTP acorde. La función _rechazar_si_fallo traduce el ResultadoCompra en HTTPException. Ejemplo de respuesta (422):

   ```
   { "detail": "El parque está cerrado en la fecha seleccionada" }
   ```

   Una compra exitosa devuelve RespuestaCompra (201), por ejemplo con el campo mensaje: "Compraste 2 entradas para el 29/10/2026. Total: $20.000"

Situación	Dónde se detecta	Respuesta
Falta el encabezado `X-Usuario-Email`	`obtener_usuario_actual`	401 «Debe iniciar sesión»
Usuario no registrado	`ServicioCompra.comprar` (`denegada`)	403 «Debe estar registrado para comprar entradas»
Regla de negocio incumplida (fecha, cantidad, edades, pase, pago, límite por fecha)	`ValidadorCompra`, `_verificar_limite_por_fecha`	422 con el mensaje de la regla
Mercado Pago no permite crear el pago	Captura de `PasarelaPagoError` en `comprar`	422 «No se pudo iniciar el pago. Intente nuevamente»; la compra no se registra
No se puede verificar el pago	Captura de `PasarelaPagoError` en `confirmar_pago`	422 «No se pudo verificar el pago con Mercado Pago»
Pago sin compra asociada	`confirmar_pago`	422 «No se encontró la compra de este pago»
Compra de otro usuario	`confirmar_pago` (`denegada`)	403 «La compra pertenece a otro usuario»
Pago no aprobado	`confirmar_pago`	422 «El pago no fue aprobado por Mercado Pago»

5. Comentarios y Documentación

Comentarios de línea (#)

Se utilizan para aclarar detalles de implementación que no son inmediatamente obvios ejemplo: calendario_parque.py: FERIADOS_CERRADO = {(12, 25), (1, 1)} # (mes, día).

Docstrings (""""Docstring"""")

Funcionan como comentarios de bloque formales.

Se utilizan inmediatamente después de la definición de una clase o función para describir su propósito y su responsabilidad principal ejemplo: ServicioCompra.confirmar_pago: «Verifica el pago con Mercado Pago y, si está aprobado, envía el mail (una sola vez).»

6. Librerías

Componentes de la API (Backend)

- **FastAPI**: framework web principal; maneja rutas, dependencias, validación de peticiones y CORS.
- **Pydantic**: define y valida los esquemas de entrada y salida (PedidoCompra, RespuestaCompra).
- **httpx**: cliente HTTP usado por PasarelaMercadoPago para comunicarse con la API de Mercado Pago
- **python-dotenv**: carga las variables de configuración (secretos, URL) desde un archivo .env.
- **segno**: genera el código QR en PNG que se incluye en el mail de confirmación

Pruebas y Desarrollo

- **pytest:** framework de pruebas para escribir y ejecutar los tests, con fixtures, parametrize y monkeypatch.
- **httpx.MockTransport y unittest.mock.MagicMock**: simulan Mercado Pago y el servidor SMTP en las pruebas unitarias
- **fastapi.testclient.TestClient:** permite probar las rutas HTTP sin levantar un servidor